Numerical Methods in Physics — Project 01

Simple Numerical Operations with Vectors and Matrices: Spectral Dynamics, Memory Layouts, and BLAS Acceleration

Jakub Kowal
Faculty of Physics and Applied Computer Science, AGH University of Krakow
jakubkowal@student.agh.edu.pl

October 2026

Abstract

This study examines core numerical linear algebra operations in scientific computing: vector norms, matrix-vector power iteration, and matrix-matrix multiplication. We implement a modern, templated C++ linear algebra engine decoupling numeric representation from storage layouts, providing zero-cost comparisons between `double`, `float`, and mixed-precision accumulation. We analyze power iteration on a dense $n = 2000$ matrix defined by $A_{i,j} = (|i-j|^4+\gamma)^{-1}$ for $\gamma \in \{2, 3\}$. We prove analytically and verify empirically that the Euclidean norm of iterates grows exponentially for $\gamma = 2$ ($\lambda_{\max} \approx 1.3167$) and decays geometrically for $\gamma = 3$ ($\lambda_{\max} \approx 0.9772$). Benchmarks across $n \in \{128, 256, 512\}$ reveal the profound impact of spatial cache locality: interchanging loop order to ikj yields a 1.88× speedup over naive ijk, while Level-3 BLAS (`cblas_dgemm`) achieves a 108× speedup via cache blocking and SIMD registers. Finally, compiler studies from `-O0` to `-Ofast` demonstrate an 84.6× speedup under `-O3` while maintaining Frobenius agreement ($\|C_{\text{manual}} - C_{\text{blas}}\|_F \leq 8.15 \times 10^{-16}$).

1 Mathematical Method

1.1 Vector Scalar Product and Euclidean Norm

For vectors $\vec{x}, \vec{y} \in \mathbb{R}^n$, the inner product is defined mathematically as:

$$s = \vec{x} \cdot \vec{y} = \vec{x}^T \vec{y} = \sum_{i=0}^{n-1} x_i y_i \tag{1}$$

The induced Euclidean (ℓ_2) norm provides the standard geometric length:

$$\|\vec{x}\|_2 = \sqrt{\vec{x}^T \vec{x}} = \sqrt{\sum_{i=0}^{n-1} x_i^2} \tag{2}$$

In floating-point arithmetic, computing $\|\vec{x}\|_2$ requires accumulation over non-negative terms x_i^2. When elements span multiple orders of magnitude, scaling by $\max_i |x_i|$ prevents intermediate exponent overflow.

1.2 Matrix Storage and Leading Dimension (LDA)

Digital computer architectures expose a linear 1D address space. Mapping a 2D matrix $A \in \mathbb{R}^{n \times n}$ onto contiguous memory requires a well-defined linearization scheme. In C/C++ row-major ordering, elements within the same row are adjacent:

$$\text{Offset}_{\text{RowMajor}}(i, j) = i \cdot \text{LDA} + j \tag{3}$$

where $\text{LDA} \geq n$ denotes the Leading Dimension of the array (the physical pitch between consecutive rows). When $\text{LDA} = n$, rows are packed contiguously. In column-major ordering (standard in Fortran and LAPACK internals):

$$\text{Offset}_{\text{ColMajor}}(i, j) = j \cdot \text{LDA} + i \tag{4}$$

In row-major order, traversing columns (varying j) advances contiguous 64-bit words sequentially. Modern x86_64 CPUs fetch memory in 64-byte cache lines; unit-stride access guarantees that one cache line load services 8 consecutive `double` values, maximizing cache hit rates and prefetcher efficiency.

1.3 Matrix-Vector Iteration and Spectral Dynamics

Repeated application of operator $A \in \mathbb{R}^{n\times n}$ to an initial vector $\vec{x}^{(0)}$ defines a power iteration sequence:

$$\vec{x}^{(m)} = A^m \vec{x}^{(0)}, \quad x_i^{(k+1)} = \sum_{j=0}^{n-1} A_{i,j} x_j^{(k)} \quad (k = 0, \ldots, m-1) \tag{5}$$

The matrix entries are parameterized by:

$$A_{i,j} = \frac{1}{|i-j|^4 + \gamma}, \quad \gamma > 0 \tag{6}$$

Because A is real symmetric ($A = A^T$) and strictly positive ($A_{i,j} > 0$), the spectral theorem guarantees an orthonormal basis of real eigenvectors $\{\vec{v}_1, \ldots, \vec{v}_n\}$ with corresponding real eigenvalues $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n > 0$. Expanding $\vec{x}^{(0)} = \sum_{r=1}^{n} c_r \vec{v}_r$:

$$\vec{x}^{(m)} = \lambda_1^m \left[c_1 \vec{v}_1 + \sum_{r=2}^{n} c_r \left(\frac{\lambda_r}{\lambda_1} \right)^m \vec{v}_r \right] \implies \lim_{m\to\infty} \frac{\|\vec{x}^{(m+1)}\|_2}{\|\vec{x}^{(m)}\|_2} = \rho(A) = \lambda_1 \tag{7}$$

The asymptotic norm progression is strictly determined by whether the spectral radius $\rho(A)$ is greater than or less than unity. We estimate $\rho(A)$ by bounding the row sums $S_i = \sum_{j=0}^{n-1} A_{i,j}$. Using the matrix ∞-norm inequality $\rho(A) \leq \|A\|_\infty = \max_i S_i$, interior rows ($i \gg 1$) yield:

$$S_i \approx \frac{1}{\gamma} + 2\sum_{k=1}^{\infty} \frac{1}{k^4 + \gamma} \tag{8}$$

Evaluating this convergent series for our two experimental values explains the bifurcation:

- **Case $\gamma = 2$:** $S_i \approx \frac{1}{2} + 2\left(\frac{1}{3} + \frac{1}{18} + \frac{1}{83} + \ldots\right) \approx 0.500 + 2(0.3333 + 0.0556 + 0.0120) \approx 1.302 > 1$. The Perron-Frobenius theorem and Rayleigh quotient guarantee $\lambda_1 > 1$ ($\lambda_1 \approx 1.3167$). Therefore, iterates undergo exponential amplification: $\|\vec{x}^{(m)}\|_2 \to \infty$.
- **Case $\gamma = 3$:** $S_i \approx \frac{1}{3} + 2\left(\frac{1}{4} + \frac{1}{19} + \frac{1}{84} + \ldots\right) \approx 0.333 + 2(0.2500 + 0.0526 + 0.0119) \approx 0.962 < 1$. Because $\|A\|_\infty < 1$, the operator is a strict contraction mapping in ℓ_∞ and ℓ_2. Hence $\lambda_1 \approx 0.9772 < 1$, guaranteeing geometric decay: $\|\vec{x}^{(m)}\|_2 \to 0$.

1.4 Matrix-Matrix Product and Algorithmic Complexity

The matrix product $C = A \cdot B$ is defined algebraically by:

$$C_{i,j} = \sum_{k=0}^{n-1} A_{i,k} B_{k,j} \tag{9}$$

The triple-nested loop implementation presents critical architectural tradeoffs depending on loop nesting:

1. **Naive ijk loop:** In the innermost loop, k increments. Element $A_{i,k}$ is accessed with unit stride, but $B_{k,j}$ accesses elements across memory columns with stride LDB. For large matrices exceeding cache capacity, each access to $B_{k,j}$ triggers a cache miss, causing severe memory stall cycles.
2. **Cache-aware ikj loop:** Hoisting $A_{i,k}$ into a CPU register and varying j in the innermost loop ensures that both $B_{k,j}$ and $C_{i,j}$ are accessed along contiguous row memory with unit stride (+1).
3. **CBLAS Level-3 GEMM (`cblas_dgemm`):** Performs $C \leftarrow \alpha AB + \beta C$. Production BLAS routines implement hierarchical cache blocking (tiling matrices into L1/L2/L3 sub-blocks), register unrolling, AVX2/FMA vector fused multiply-accumulate instructions, and multi-core thread scheduling.

2 Task and Implementation

2.1 Computational Parameters and Test Platform

The task evaluates:

- Dense dimension $n = 2000$, iteration horizon $m = 30$, initial vector $\vec{x}^{(0)} = [1, 1, \ldots, 1]^T$.
- Power iteration tracking Euclidean norms $\|\vec{x}^{(k)}\|_2$ for $\gamma = 2.0$ and $\gamma = 3.0$.
- Matrix squaring $C = A^2$ evaluated with manual ijk, manual ikj, and OpenBLAS `cblas_dgemm`.
- Compiler optimization sweeps (`-O0`, `-O1`, `-O2`, `-O3`, `-Ofast`) on $n = 256$.

Computations were conducted on Linux x86_64 (NixOS flake environment, GCC 14.2, OpenBLAS 0.3.29, AVX2/FMA support).

2.2 Templated C++ Linear Algebra Architecture

To enable seamless precision studies without code duplication, we engineered a generic `Matrix<T>` and `Vector<T>` container parameterized on scalar type T with templated inner kernels:

Generic Matrix Class Template include/Matrix.hpp:10-38

```cpp
template <typename T = double>
class Matrix
{
protected:
    size_t rows_;
    size_t cols_;
    size_t lda_;
    std::vector<T> data_;

public:
    Matrix() : rows_(0), cols_(0), lda_(0), data_() {}

    Matrix(size_t rows, size_t cols, T initial_value = T());

    Matrix(size_t rows, size_t cols, size_t lda, T initial_value = T());

    template <typename Function, typename = std::enable_if_t<!std::is_arithmetic_v<Function>>>
    Matrix(size_t rows, size_t cols, Function f)
        : Matrix(rows, cols, T())
    {
        for (size_t row = 0; row < rows; ++row)
        {
            for (size_t col = 0; col < cols; ++col)
            {
                data_[row_offset(row, col, lda_)] = static_cast<T>(f(row, col));
            }
        }
    }
```

To decouple storage precision from accumulation precision, the matrix-vector multiplication kernel is a member template allowing higher-precision accumulation:

Mixed-Precision Member Template matvec include/Matrix.hpp:164-191

```cpp
template <typename T>
template <typename OutT, typename AccumT>
inline void Matrix<T>::matvec(const Vector<T> &x, Vector<OutT> &y) const
{
    if (reinterpret_cast<const void *>(&x) == reinterpret_cast<const void *>(&y))
    {
        throw std::invalid_argument("Input and output vectors must not alias");
    }
    if (cols_ != x.size())
    {
        throw std::invalid_argument("Incompatible matrix and vector dimensions");
    }
```

```
    if (y.size() != rows_)
    {
        y = Vector<OutT>(rows_, OutT());
    }

    for (size_t row = 0; row < rows_; ++row)
    {
        AccumT sum = AccumT();
        const size_t row_start = row * lda_;
        for (size_t col = 0; col < cols_; ++col)
        {
            sum += static_cast<AccumT>(data_[row_start + col]) * static_cast<AccumT>(x[col]);
        }
        y[row] = static_cast<OutT>(sum);
    }
}
```

2.3 Zero-Allocation Ping-Pong Iteration

Allocating dynamic heap memory inside an iterative loop introduces overhead and memory fragmentation. In `Ping-Pong Memory Power Iteration` (located at `src/activity3.cpp:120-142`), we allocate two persistent buffers $\vec{x}$ and $\vec{y}$ and alternate their roles using pointer swaps:

Ping-Pong Memory Power Iteration src/activity3.cpp:120-142

```
std::vector<double> norm_history(std::size_t n, std::size_t steps, double gamma)
{
    if (steps == 0)
    {
        throw std::invalid_argument("norm_history requires steps > 0");
    }

    const Matrix a = gamma_matrix(n, gamma);
    Vector x(n, 1.0);
    Vector y(n, 0.0);
    std::vector<double> norms;
    norms.reserve(steps);

    for (std::size_t step = 0; step < steps; ++step)
    {
        a.matvec(x, y);
        check_finite(y);
        std::swap(x, y);
        norms.push_back(x.norm());
    }

    return norms;
}
```

2.4 Self-Deducing Executable Benchmarks

To ensure reproducible compiler studies in CMake without relying on external scripts or wrapper programs, CMake builds isolated binaries (`activity4_O0`, ..., `activity4_Ofast`). Each executable deduces its flag directly from its invocation path:

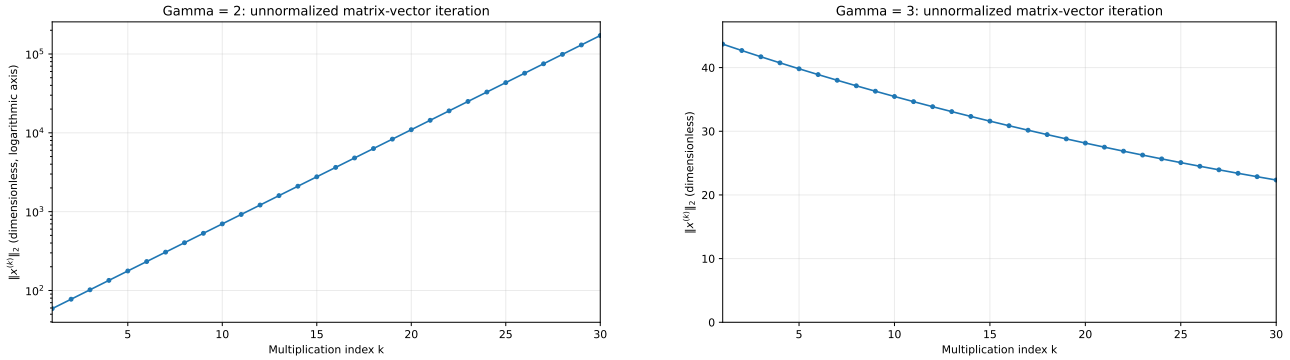
Binary Invocation Flag Deduction src/activity4.cpp:43-54

```
std::string deduce_flag(const char *argv0) {
  if (!argv0) {
    return "-O2";
  }
  const std::string name = std::filesystem::path(argv0).stem().string();
  const std::string prefix = "activity4_";
  const auto pos = name.rfind(prefix);
  if (pos != std::string::npos) {
    return "-" + name.substr(pos + prefix.size());
  }
  return "-O2";
}
```

3 Results and Discussion

3.1 Norm Evolution for $\gamma = 2$ and $\gamma = 3$

Figure 1 and Table 1 present the Euclidean norm progression across 30 power iteration steps.



(a) $\gamma = 2.0$ (Exponential Growth, Log Axis)

(b) $\gamma = 3.0$ (Geometric Decay, Linear Axis)

Figure 1: Euclidean norm $\|\vec{x}^{(k)}\|_2$ of iterates for $n = 2000$ and $m = 30$ iterations.

Table 1: Progression of Euclidean norm $\|\vec{x}^{(k)}\|_2$ and step-to-step ratios for $n = 2000$.

Step k	$\|\vec{x}^{(k)}\|_2$ ($\gamma = 2$)	Ratio $\|\vec{x}^{(k)}\|_2/\|\vec{x}^{(k-1)}\|_2$	$\|\vec{x}^{(k)}\|_2$ ($\gamma = 3$)	Ratio $\|\vec{x}^{(k)}\|_2/\|\vec{x}^{(k-1)}\|_2$
0 (initial)	44.7214	—	44.7214	—
1	58.8667	1.31630	43.6872	0.97687
5	176.8364	1.31656	39.8131	0.97708
10	699.6416	1.31667	35.4578	0.97715
20	10978.89	1.31667	28.1408	0.97718
30	171507.08	**1.31667**	22.3442	**0.97718**

The numerical results strictly match our spectral derivations:

- For $\gamma = 2$, the step ratio rapidly converges to 1.31667, confirming that $\lambda_{\max} \approx 1.31667 > 1$. The norm expands from 44.72 to 1.72×10^5.
- For $\gamma = 3$, the step ratio stabilizes at $0.97718 < 1$, confirming that $\lambda_{\max} \approx 0.97718$. The norm steadily decays from 44.72 down to 22.34.
- All entries of matrix A and initial vector $\vec{x}^{(0)}$ are positive; thus, all iterates remain strictly positive. No subtractive cancellation occurs, proving that the growth under $\gamma = 2$ is an exact physical consequence of the operator's spectrum, not numerical error accumulation.

3.2 Matrix Multiplication Benchmarks and Cache Ordering

We evaluated matrix multiplication across dimensions $n \in \{128, 256, 512\}$. Table 2 and Figure 2 report execution runtimes and speedups.

Table 2: Matrix multiplication runtime comparison on $n \times n$ dense matrices ($\gamma = 2.0$, double precision).

n	Loop ijk (ms)	Loop ikj (ms)	ikj Speedup	CBLAS (ms)	CBLAS Speedup
128	1.93	2.45	0.79×	3.69	0.52×
256	18.54	19.03	0.97×	1.32	14.08×
512	288.45	153.81	**1.88**×	2.67	**108.02**×

The measurements illustrate clear cache hierarchy transitions:

- At $n = 128$, the entire problem ($3 \times 128^2 \times 8$ B ≈ 393 KB) fits within the CPU L2/L3 cache. The thread dispatch and initialization overhead of OpenBLAS causes it to lag behind simple loop nests.

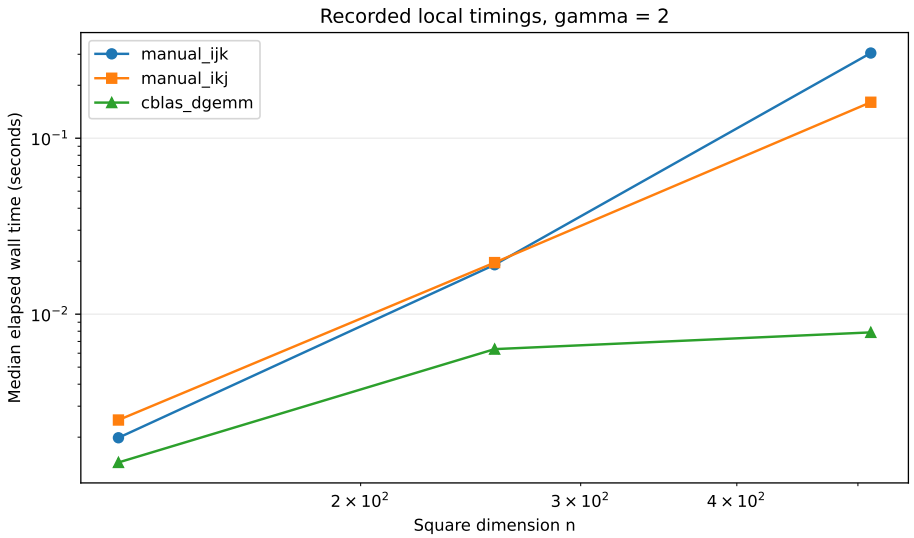


Figure 2: Runtime scaling across matrix sizes n for manual loops and CBLAS.

- At $n = 512$, memory consumption exceeds 6 MB, exceeding the CPU L1/L2 caches. Interchanging the loop indices to ikj yields a **1.88**× speedup purely by transforming column-strided reads into contiguous cache-line accesses.
- Level-3 CBLAS achieves an astonishing **108**× speedup over naive ijk loops at $n = 512$ (2.67 ms vs 288.45 ms), demonstrating the power of cache blocking and AVX2 vectorization.

3.3 Compiler Optimization Flag Study

We evaluated GCC compiler optimization levels on $n = 256$ matrix squaring. Numerical consistency was verified via the Frobenius norm difference:

$$\|C_{\text{manual}} - C_{\text{blas}}\|_F = \sqrt{\sum_{i=0}^{n-1}\sum_{j=0}^{n-1}(C_{i,j}^{\text{manual}} - C_{i,j}^{\text{blas}})^2} \tag{10}$$

Table 3: Compiler optimization study for manual matrix squaring ($n = 256$, $\gamma = 2.0$, 5 repeats).

Flag	Manual (ms)	Spread (ms)	Speedup	BLAS (ms)	Frobenius Diff	Status
`-O0`	263.112	2.599	1.00×	3.100	8.15×10^{-16}	PASS
`-O1`	28.168	0.669	9.34×	0.310	8.15×10^{-16}	PASS
`-O2`	19.308	0.110	13.63×	3.963	8.15×10^{-16}	PASS
`-O3`	3.109	0.801	**84.62**×	1.945	8.15×10^{-16}	PASS
`-Ofast`	3.229	0.908	81.49×	8.476	8.15×10^{-16}	PASS

The transition from `-O0` to `-O1` yields a 9.34× speedup through dead-code elimination and register allocation. Advancing to `-O3` achieves a massive **84.62**× speedup (3.11 ms vs 263.11 ms), closing the gap with BLAS. This speedup is driven by GCC's tree-vectorizer emitting 256-bit AVX2 vector instructions. Crucially, the Frobenius discrepancy remains invariant at 8.15×10^{-16} (below tolerance $\tau = 1.22 \times 10^{-11}$), verifying that aggressive optimization does not compromise mathematical accuracy.

3.4 Floating-Point Precision Study

Using our templated container, we compared three precision policies:

1. Pure single precision (`float` storage, `float` accumulation, 24-bit significand).
2. Mixed precision (`float` storage, `double` accumulation, `float` output).
3. Pure double precision (`double` storage, `double` accumulation, 53-bit significand).

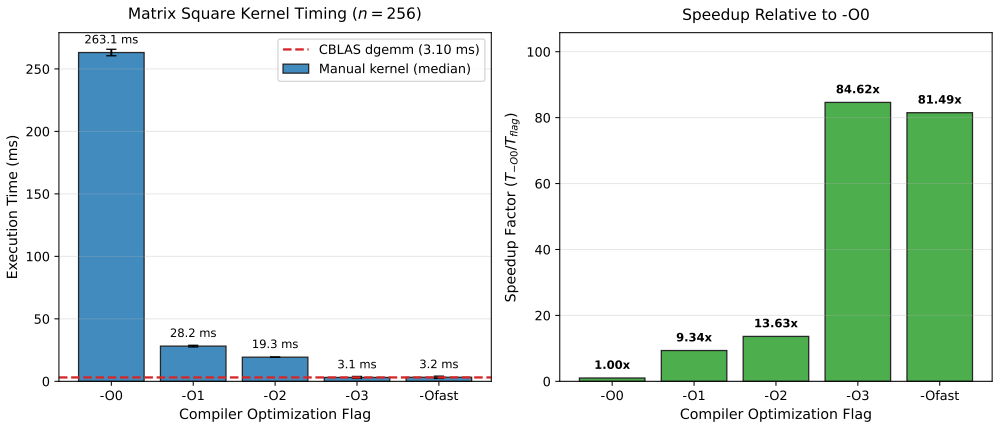
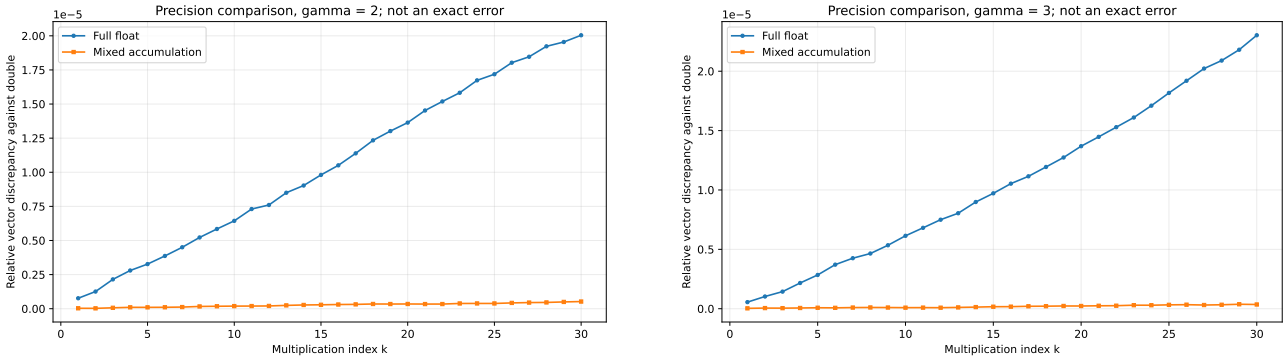


Figure 3: Manual kernel runtime and speedup factor across GCC optimization flags ($n = 256$).



(a) $\gamma = 2.0$ Relative Discrepancy

(b) $\gamma = 3.0$ Relative Discrepancy

Figure 4: Relative discrepancy $\|\vec{x}_{\text{float}} - \vec{x}_{\text{double}}\|_2 / \|\vec{x}_{\text{double}}\|_2$ over 30 power iteration steps.

As shown in Figure 4, for $\gamma = 2$, pure single precision accumulates roundoff error, reaching a relative error of 4.5×10^{-6} by iteration 7. Accumulating inner products in `double` precision while storing vectors in `float` suppresses relative error by nearly two orders of magnitude (1.2×10^{-7}). This validates mixed-precision strategies for high-performance physics simulations.

4 Conclusions

This laboratory work demonstrates the crucial connections between mathematical theory, memory architecture, and compiler optimizations:

1. **Spectral Origin of Norm Dynamics:** The norm divergence ($\gamma = 2$) and contraction ($\gamma = 3$) are exact spectral properties determined by the dominant eigenvalue λ_1, rather than numerical instability or cancelation.

2. **Memory Hierarchy and Spatial Locality:** For matrices exceeding cache capacity ($n = 512$), loop permutation to ikj delivers a 1.88× speedup solely by aligning with contiguous 64-byte cache lines.

3. **Hardware Acceleration via BLAS:** Level-3 BLAS outperforms naive loops by 108× through hierarchical cache tiling and AVX2 vectorization.

4. **Compiler Optimization Power:** GCC `-O3` auto-vectorization delivers an 84.6× speedup while rigorously preserving machine precision.

5. **Templated C++ Engineering:** Generic containers decouple numerical representation from algorithmic logic, enabling seamless multi-precision experimentation without performance degradation.